

Manual de Despliegue en Ubuntu Server

Sistema de Monitoreo de Red Ubiquiti
Backend colector · InfluxDB · Dashboard React · Notificador Telegram

Guía completa de instalación
Archivos necesarios · Archivos excluidos · Puesta en marcha · Verificación

Guía	Despliegue del proyecto en Ubuntu Server
Contenedores	InfluxDB 1.8 · Colector · Dashboard · Telegram
Requiere	Docker + Docker Compose en Ubuntu Server

1. Introducción

El proyecto **ubiquiti-monitor** es un sistema de monitoreo de red que consulta la API de Ubiquiti NMS, almacena la información en InfluxDB y la presenta en un panel web (React servido por una API Flask). Se ejecuta mediante **4 contenedores Docker** orquestados con Docker Compose:

- **influxdb** — base de datos de series temporales (imagen oficial `influxdb:1.8`).
- **ubiquiti-backend** — colector que consulta la API de Ubiquiti cada `POLLING_INTERVAL` segundos.
- **dashboard** — API Flask + aplicación React (compilada dentro de Docker).
- **telegram-notifier** — bot que envía alertas de juntas caídas a Telegram.

Los cuatro contenedores comparten la red interna `monitoring` y el puerto del dashboard se publica al exterior (por defecto el **5000**).

El build de React se genera dentro de Docker (multi-stage), por lo que **no es necesario subir las carpetas de dependencias ni el build compilado**.

2. Requisitos previos

Antes de empezar asegúrate de tener disponible lo siguiente:

- **Servidor virtual:** Ubuntu Server 22.04 LTS o 24.04 LTS (64-bit), mínimo 2 vCPU, 4 GB RAM y 20 GB de disco.
- **Docker Engine** y **Docker Compose** v2 instalados (se instalan en la sección 6).
- **Acceso SSH** al servidor y a la máquina de origen (para transferir archivos con `scp`).
- **Credenciales de Ubiquiti NMS:** token de la API y URL del NMS.
- **Credenciales de Telegram** (opcional): token del bot y chat ID, si se usan alertas.
- **Archivos de datos:** los libros de Excel (`base_operacion.xlsx`, `base_instalaciones.xlsx` y `Grupo 1 - Juntas de Internet.xlsx`).

La arquitectura orientativa del servidor:

```
Internet | v [Ubuntu Server] | +-- puerto 5000 --> ubiquiti-dashboard (Flask + React) | +-- red interna "monitoring" (bridge) +-- influxdb:1.8 (solo interno) +-- ubiquiti-backend (solo interno) +-- telegram-notifier (salida a Telegram)
```

3. Archivos que DEBES subir

A continuación se listan los archivos necesarios. Todos se ubican en la raíz del proyecto. En la máquina de origen (donde hoy funciona el sistema) estos archivos están en la carpeta del proyecto.

Estructura mínima a copiar:

```

ubiquiti-monitor/ |-- docker-compose.yml # Orquestación de los 4 servicios |-- .env
# Credenciales y configuración (se crea en el servidor) |-- .env.example # Plantilla
de configuración |-- Dockerfile # Imagen del colector (ubiquiti-backend) |--
Dockerfile.dashboard # Imagen de Flask + React (build multi-stage) |--
Dockerfile.telegram # Imagen del bot de Telegram |-- config.py # Configuración leída
por el backend y la API |-- api.py # API Flask (dashboard) |-- ubiquiti_backend.py #
Colector (bucle principal) |-- telegram_notifier.py # Bot de alertas Telegram |--
requirements.txt # Dependencias Python (pinned) |-- excel/ # CARPETA con los 3
libros de datos (obligatoria) | |-- base_operacion.xlsx | |--
base_instalaciones.xlsx | `-- Grupo 1 - Juntas de Internet.xlsx |-- data/ # Datos en
runtime (usuarios, estado, tickets) * |-- logs/ # Logs en runtime * `-- dashboard/ #
Código fuente de la app React |-- package.json |-- package-lock.json |-- index.html
|-- vite.config.js |-- public/ # (imágenes estáticas) `-- src/ # Componentes React
|-- api.js |-- App.jsx |-- main.jsx |-- App.css `-- ... (páginas y componentes)

```

* Las carpetas `data/` y `logs/` las crea Docker automáticamente. Solo las subes si quieres **migrar datos existentes** (por ejemplo `usuarios` o `tickets`). Se crean vacías en el servidor y se llenan en runtime.

Archivos de datos privados (obligatorios, no están en Git):

Los tres libros de Excel se agrupan en una carpeta **excel/** dentro del proyecto. El código y docker-compose quedaron configurados para leerlos desde ahí:

```

excel/ |-- base_operacion.xlsx # Mapeo operación (backend + dashboard) |--
base_instalaciones.xlsx # Mapeo instalaciones (backend) `-- Grupo 1 - Juntas de
Internet.xlsx # Mapeo de grupos (backend)

```

INFORMACIÓN: Sí, el archivo **Grupo 1 - Juntas de Internet.xlsx** es necesario: es el que asigna el grupo (ej. Grupo 1) a cada junta, dato que se muestra en el panel. Sin él las juntas no tendrán grupo asignado. El nombre debe conservar el texto Grupo N: el sistema infiere el nombre del grupo desde el nombre del archivo.

ATENCIÓN: No cambies los nombres de estos 3 archivos ni los muevas fuera de `excel/`: el código y el mapeo de docker-compose esperan esas rutas exactas.

4. Archivos que NO debes subir

Estos elementos **deben quedar excluidos** de la transferencia. Son dependencias locales, builds compilados, cachés o datos temporales que Docker regenera por sí solo:

- `dashboard/node_modules/` — dependencias de npm locales; el build en Docker las instala.
- `dashboard/dist/` — build de React compilado; se regenera dentro del `Dockerfile.dashboard`.
- `__pycache__/` y `*.pyc` — caché de Python.
- `.git/` — historial de Git (opcional subirlo si lo deseas, pero no es necesario para ejecutar).
- `.vscode/`, `.idea/` — configuración de IDE.
- `*.log/` `ubiquiti_monitor.log` — logs locales de desarrollo.
- `.env` de tu máquina de desarrollo (el del servidor se crea aparte, ver sección 7).
- `data/users.json`, `telegram_sent_alerts.json`, etc. — solo si NO quieres migrar estado.

- Thumbs.db, .DS_Store — archivos del sistema operativo.

ATENCIÓN: No subas la carpeta node_modules/ ni el dist/: además de compilarse en Docker, están excluidas por el archivo .dockerignore, por lo que su presencia no aporta nada y ralentiza la transferencia.

Resumen rápido para recordar:

SUBIR	NO SUBIR
docker-compose.yml	node_modules/
.env (creado en el servidor)	dist/
Dockerfile* (3 archivos)	__pycache__/
config.py, api.py, ubiquiti_backend.py, telegram_notifier.py	.git/
requirements.txt	.vscode/, .idea/
dashboard/ (sin node_modules ni dist)	*.log
excel/ (los 3 libros de datos)	.DS_Store, Thumbs.db

5. Archivo de tickets (nombre obligatorio)

El módulo de tickets puede trabajarse de dos formas: **cargándolo manualmente** o **subiéndolo desde el panel** (botón Cargar Excel en la página de Tickets).

Regla de nombres: el sistema busca el archivo en este orden:

- Prioridad 1: data/tickets_juntas.xlsx (el nombre exacto que usa la carga desde el panel).
- Prioridad 2: cualquier archivo que empiece por DetalleTickets_ y termine en .xlsx (se detecta automáticamente).

Cuando subes el archivo desde el panel web, el servidor siempre lo guarda como data/tickets_juntas.xlsx. Si lo copias manualmente al servidor, usa ese mismo nombre para que se encuentre sin tocar código.

ATENCIÓN: El archivo NO se coloca en la carpeta excel/: va en data/. Además debe tener la columna Ticket Estado y Código Operador para que el panel muestre los tickets abiertos.

6. Transferencia de archivos (SCP)

El método recomendado es usar **SCP** (viene integrado en OpenSSH de Windows y Linux). Primero crea la estructura de carpetas en el servidor:

```
# En el servidor Ubuntu (por SSH): mkdir -p ~/ubiquiti-monitor cd ~/ubiquiti-monitor
```

Una vez creada, transfiere el contenido desde tu máquina de origen. Ejemplo desde **Windows PowerShell** abierto en la carpeta del proyecto:

```
# Desde Windows PowerShell (en la carpeta del proyecto): $server =  
"usuario@IP_DEL_SERVIDOR" # 1) Todos los archivos de la raíz (sin dependencias ni  
Git) scp docker-compose.yml .env.example Dockerfile Dockerfile.dashboard  
Dockerfile.telegram ` config.py api.py ubiquiti_backend.py telegram_notifier.py  
requirements.txt ` "$server:~/ubiquiti-monitor/" # 2) La carpeta excel/ con los 3  
libros de datos scp -r excel "$server:~/ubiquiti-monitor/" # 3) La carpeta dashboard  
completa excepto node_modules y dist scp -r dashboard "$server:~/ubiquiti-monitor/"
```

Alternativa desde **Linux/Mac**:

```
scp docker-compose.yml .env.example Dockerfile* \\  
config.py api.py  
ubiquiti_backend.py telegram_notifier.py requirements.txt \\  
user@IP:/home/user/ubiquiti-monitor/ scp -r excel  
user@IP:/home/user/ubiquiti-monitor/ scp -r dashboard  
user@IP:/home/user/ubiquiti-monitor/
```

Si usas una herramienta gráfica (WinSCP, FileZilla, MobaXterm) transfiere los mismos archivos, respetando la lista de la sección 4.

7. Instalar Docker en Ubuntu Server

Conecta por SSH y ejecuta lo siguiente (script oficial de Docker). El proceso pide contraseña de sudo al final para añadir tu usuario al grupo docker:

```
# Instalar Docker Engine + Compose plugin curl -fsSL https://get.docker.com | sh #  
Dar permisos para usar docker sin sudo (reconectar después) sudo usermod -aG docker  
$USER exit # vuelve a conectarte por SSH
```

Comprueba la instalación:

```
docker --version docker compose version
```

Resultado esperado: versiones de Docker (24+) y Compose (v2.x). Si `docker compose version` no existe, instala el plugin:

```
sudo apt update sudo apt install -y docker-compose-plugin
```

8. Crear el archivo de configuración (.env)

El proyecto se configura con variables de entorno almacenadas en un archivo `.env`. **No cuenta el `.env` de tu máquina de desarrollo:** crea uno limpio en el servidor. El más seguro es generar las contraseñas en el propio servidor. Edita el archivo:

```
cd ~/ubiquiti-monitor nano .env
```

Copia y rellena con tus valores reales:

```
# ===== API UBIQUITI ===== UBIQUITI_API_URL=https://tu-nms.inred.com.co/nms/api/v2.1  
UBIQUITI_API_TOKEN=TU_TOKEN_REAL UBIQUITI_VERIFY_SSL=true # ===== INFLUXDB =====  
INFLUXDB_HOST=influxdb INFLUXDB_PORT=8086 INFLUXDB_ADMIN_USER=admin  
INFLUXDB_ADMIN_PASSWORD=una_contrasena_muy_segura INFLUXDB_DATABASE=monitoreo #  
===== DASHBOARD ===== DASHBOARD_PORT=5000 SECRET_KEY=clave_secreta_aleatoria_larga  
DASHBOARD_USER=admin DASHBOARD_PASSWORD=clave_del_panel # ===== MONITOREO =====  
POLLING_INTERVAL=300 # ===== TELEGRAM =====  
TELEGRAM_BOT_TOKEN=123456789:TU-BOT-TOKEN TELEGRAM_CHAT_ID=123456789  
TELEGRAM_CHECK_INTERVAL=300 # ===== DOCKER =====  
COMPOSE_PROJECT_NAME=ubiquiti-monitor ENVIRONMENT=production
```

ATENCIÓN: Nunca guardes el archivo `.env` en un repositorio público ni lo compartas: contiene tokens y contraseñas. Está excluido por `.gitignore` y `.dockerignore`.

Genera una clave secreta del dashboard aleatoria con:

```
# recomendado para SECRET_KEY openssl rand -hex 32
```

9. Puesta en marcha

Con los archivos en su sitio y el `.env` creado, validamos la configuración y levantamos TODOS los servicios:

```
cd ~/ubiquiti-monitor # Validar la sintaxis de docker-compose docker compose config
# Construir imágenes y arrancar en segundo plano docker compose up -d --build
```

La primera ejecución descarga las imágenes base y compila la app React; puede tardar varios minutos.

Para ver el estado y los logs en vivo:

```
docker compose ps docker compose logs -f ubiquiti-backend docker compose logs -f
dashboard docker compose logs -f telegram-notifier
```

10. Verificación del despliegue

Comprueba que los 4 contenedores están **Up** y **healthy**:

```
docker compose ps
```

Salida esperada (estado **Up** / **healthy**):

```
NAME IMAGE STATUS PORTS ubiquiti-dashboard ubiquiti-monitor-dashboard Up (healthy)
0.0.0.0:5000->5000/tcp ubiquiti-influxdb influxdb:1.8 Up (healthy) 8086/tcp
ubiquiti-monitor ubiquiti-monitor-backend Up (healthy) ubiquiti-telegram
ubiquiti-monitor-telegram Up
```

Comprobar el endpoint de salud del dashboard desde tu navegador o con curl:

```
# Desde el servidor: curl http://localhost:5000/api/health # Desde otra máquina:
curl http://IP_DEL_SERVIDOR:5000/api/health
```

Respuesta esperada:

```
{"status": "ok", "timestamp": "2026-08-15T10:00:00.000000"}
```

Abrir el panel web: `http://IP_DEL_SERVIDOR:5000` e iniciar sesión con `DASHBOARD_USER` / `DASHBOARD_PASSWORD`.

INFORMACIÓN: El backend consulta la API de Ubiquiti cada `POLLING_INTERVAL` (por defecto 300 s). Es normal que las primeras métricas tarden unos minutos en aparecer.

11. Reconstruir y reiniciar servicios

Comandos útiles para el día a día:

```
# Ver logs de un servicio docker compose logs -f ubiquiti-backend # Reconstruir
después de cambiar código (api.py, ubiquiti_backend.py, dashboard/src) docker
compose up -d --build ubiquiti-backend docker compose up -d --build dashboard #
Reiniciar solo un servicio docker compose restart dashboard # Detener y arrancar
todo docker compose down docker compose up -d # Detener y ELIMINAR los volúmenes
(borrar TODA la base de datos) -- ¡CUIDADO! docker compose down -v
```

ATENCIÓN: `docker compose down -v` borra el volumen de InfluxDB y **toda** la base de datos histórica. Úsalo solo si quieres empezar de cero.

12. Solución de problemas frecuentes

11.1 El dashboard no responde en el puerto 5000

- Revisa que el firewall de Ubuntu permita el puerto: `sudo ufw allow 5000/tcp`.
- Verifica el estado del contenedor: `docker compose ps`.
- Revisa los logs: `docker compose logs -f dashboard`.

11.2 El colector no conecta con InfluxDB

- Confirma que las variables `INFLUXDB_HOST`, `INFLUXDB_PORT` y el nombre de BD en `.env` sean correctos.
- Comprueba los logs: `docker compose logs -f ubiquiti-backend`.
- El host de InfluxDB **debe ser** `influxdb` (nombre del servicio en la red interna), no la IP del servidor.

11.3 Error 401 / credenciales de InfluxDB

- Las credenciales del `.env` deben coincidir en los 4 servicios (se pasan a los contenedores).
- Tras cambiar el `.env`, reconstruye y reinicia: `docker compose up -d`.

11.4 El tráfico o los dispositivos no se ven en el panel

- Espera a que se complete al menos un ciclo de `POLLING_INTERVAL`.
- Confirma que Ubiquiti responde: `docker compose logs -f ubiquiti-backend`.
- Verifica el token y la URL: `UBIQUITI_API_URL` / `UBIQUITI_API_TOKEN`.

11.5 No llegan alertas de Telegram

- Valida `TELEGRAM_BOT_TOKEN` y `TELEGRAM_CHAT_ID` en el `.env`.
- Revisa logs: `docker compose logs -f telegram-notifier`.

11.6 Cambios en el código no se reflejan

Debes reconstruir la imagen (no basta con reiniciar), porque el código queda dentro de la imagen: `docker compose up -d --build`. Para el dashboard, además se recompila React.

11.7 Permisos denegados de Docker

- Ejecuta `sudo usermod -aG docker $USER` y vuelve a conectar por SSH.
- Como alternativa temporal: prefija los comandos con `sudo`.

13. Acceso al proyecto desde internet

El dashboard ya escucha en el puerto 5000 dentro del servidor. Para acceder desde **internet** tienes varias opciones, de la más sencilla a la más segura:

Opción A — Nginx como proxy inverso + HTTPS (recomendada)

Se instala Nginx en Ubuntu, que actúa como puerta de entrada: redirige el tráfico del puerto 443 (HTTPS) hacia el contenedor del dashboard (puerto 5000). Esto añade cifrado con certificado Let's Encrypt.

```
# 1) Instalar Nginx y Certbot sudo apt update sudo apt install -y nginx certbot python3-certbot-nginx # 2) Configurar un sitio (reemplaza dashboard.midominio.com por tu dominio) sudo tee /etc/nginx/sites-available/ubiquiti-dashboard > /dev/null <<'EOF' server { listen 80; server_name dashboard.midominio.com; location / { proxy_pass http://127.0.0.1:5000; proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto $scheme; } } EOF # 3) Activar el sitio y recargar sudo ln -s /etc/nginx/sites-available/ubiquiti-dashboard /etc/nginx/sites-enabled/ sudo nginx -t sudo systemctl reload nginx # 4) Emitir certificado HTTPS automático (Certbot configura todo) sudo certbot --nginx -d dashboard.midominio.com
```

ATENCIÓN: Para HTTPS con Let's Encrypt necesitas un **dominio apuntando al servidor** (registro A hacia tu IP pública). Si solo tienes IP pública, usa una IP dinámica (DDNS) como DuckDNS.

Opción B — Cloudflare Tunnel (sin abrir puertos)

Ideal si no quieres exponer ni el puerto ni necesitas dominio propio. Con `cloudflared` creas un túnel que publica el puerto 5000 bajo una URL `*.trycloudflare.com` o tu propio dominio Cloudflare, sin tocar el firewall.

```
# Instalar cloudflared y arrancar túnel temporal curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64 -o /usr/local/bin/cloudflared chmod +x /usr/local/bin/cloudflared cloudflared tunnel --url http://127.0.0.1:5000
```

Opción C — Exponer directamente el puerto 5000 (más simple, menos segura)

Abrir el puerto 5000 en el firewall: `sudo ufw allow 5000/tcp`. Funciona, pero el tráfico viaja sin cifrado (HTTP) y el panel queda accesible a cualquiera que conozca la IP. Solo apto para pruebas.

Recomendación final: usa **Nginx + Let's Encrypt** (opción A) para producción: HTTPS, nombre legible y sin exponer direcciones internas. Cloudflare Tunnel es excelente si no tienes dominio o IP fija.

14. Checklist final de despliegue

- La carpeta `excel/` está en el servidor con los 3 libros (operación, instalaciones y juntas).
- El archivo de tickets (si se usa) está como `data/tickets_juntas.xlsx`.
- El archivo `.env` está creado con credenciales reales y seguras.
- No se transfirieron `node_modules/`, `dist/`, ni cachés.
- `docker compose config` no muestra errores.

- Los 4 contenedores están **Up / healthy**.
- El endpoint `/api/health` responde `status: ok`.
- El panel web abre en `http://IP_DEL_SERVIDOR:5000` con login correcto.
- En `docker compose logs -f ubiquiti-backend` no aparecen errores de API/InfluxDB.

Fin del manual

Cualquier incidencia relacionada con la configuración de variables, credenciales o datasets se resuelve editando el archivo `.env` y ejecutando `docker compose up -d`.